MBLP131 · HAFTA 5

İşletim Sistemleri Temelleri

Hafta 5 · İşlemci Zamanlama Algoritmaları ve Süreç Eşzamanlaması

Sırada kim çalışacak ve iş parçacıkları veriyi bozmadan nasıl paylaşır?

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 4. Hafta

İş parçacıkları

Bir süreç içindeki iş parçacıkları kodu, veriyi ve açık dosyaları paylaşır; her birinin kendi yazmaçları ve yığıtı (stack) vardır.

CPU–G/Ç patlamaları

Süreç, CPU patlamaları (CPU burst) ile G/Ç beklemeleri arasında gidip gelir. Patlamaların çoğu kısa, birkaçı uzundur.

Kesintili mi, değil mi?

Kesintisiz: süreç bloklanana ya da bitene kadar işlemciyi tutar. Kesintili (preemptive): işletim sistemi işlemciyi geri alabilir. Geçişi dağıtıcı yapar.

Bir planı puanlamak

- **Dönüş** = bitiş − varış
- **Bekleme** = dönüş − patlama
- **Yanıt** = ilk çalışma − varış

Bugün: bu kararı veren algoritmalar ve zamanladığımız iş parçacıkları veri paylaştığında olanlar.

Bugünün Gündemi

1. **FCFS ve SJF** İlk gelen ilk hizmet alır, en kısa iş önce, en kısa kalan süre önce.
2. **Öncelik ve döngüsel** Açlık ve yaşlandırma; zaman dilimi; çok düzeyli kuyruklar; Linux ne yapıyor?
3. **Yarış durumları** counter++ neden güncelleme kaybedebilir; kritik bölge problemi.
4. **Eşzamanlama araçları** Peterson, atomik komutlar, muteks kilitleri, semaforlar, üretici–tüketici.
5. **Uygulama** pthreads ile C'de gerçek bir yarış ve ölçülmüş çözümü.

BÖLÜM 1

İşlemci Zamanlama Algoritmaları

Tek iş kümesi, yedi plan, tek karşılaştırma tablosu.

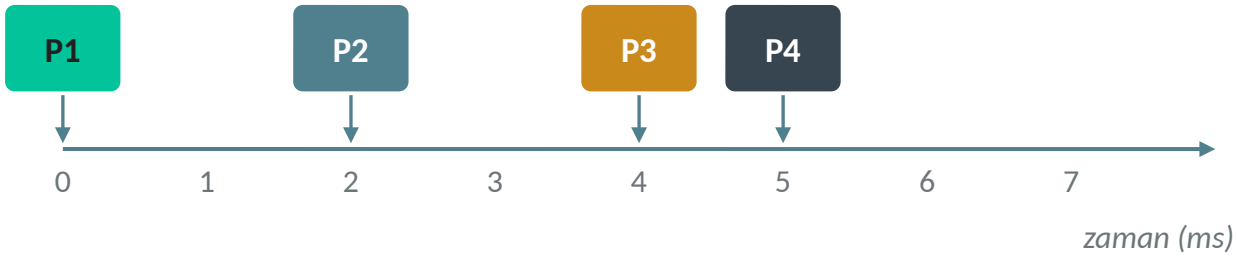
SÜREKLİ ÖRNEĞİMİZ

Her Algoritma için Tek Bir İş Kümesi

Dört süreç. Süreler milisaniye cinsinden. Öncelik numarası küçüldükçe öncelik yükselir.

Süreç	Varış zamanı	CPU patlaması	Öncelik
P1	0	7	3
P2	2	4	1
P3	4	1	4
P4	5	4	2

varışlar



Oyunun kuralları

- **Gantt şeması:** bir sıra kutu; her kutu bir CPU kullanım aralığı, altındaki işaretler zamanlar.
- **Bağlam değişimi** süresi 0 alınır (geçişleri ayrıca sayarız).
- **Eşitlikler** önce varış zamanına, sonra ada göre bozulur.
- **Tüm sayılar** küçük bir simülatörle hesaplandı: code/sched_check.py.

ALGORİTMA 1

İlk Gelen İlk Hizmet Alır (FCFS)

Hazır kuyruğu basit bir FIFO kuyruğudur: önce gelen önce çalışır; bitene ya da bloklanana kadar. Kesintisizdir (non-preemptive).



Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	7	7	0	0
P2	2	4	11	9	5	5
P3	4	1	12	8	7	7
P4	5	4	16	11	7	7
Ortalama				**8,75**	**4,75**	**4,75**

Şemayı okumak

- Varış sırası: P1 (0), P2 (2), P3 (4), P4 (5).
- Örnek, P3: dönüş 12 − 4 = 8, bekleme 8 − 1 = 7 ms.
- Basit ve 'banka kuyruğu' anlamında adil; ama kısa işler uzun işlerin arkasında uzun süre bekleyebilir.

Konvoy Etkisi: Sıra Önemlidir

Patlamaları 24, 3 ve 3 ms olan üç iş t = 0'da geliyor. Aynı işler, aynı algoritma, iki farklı varış sırası.

A sırası: uzun iş tesadüfen önde



B sırası: kısa işler tesadüfen önde



FCFS'de ortalama bekleme süresi tamamen şansa, yani işlerin tesadüfen hangi sırayla geldiğine bağlıdır.

```
Bekleme süreleri (ms)
A: P1 = 0, P2 = 24, P3 = 27
   ortalama = 51 / 3 = 17

B: P2 = 0, P3 = 3, P1 = 6
   ortalama =  9 / 3 = 3
```

Konvoy etkisi (convoy effect): kısa (G/Ç ağırlıklı) süreçler, yavaş bir kamyonun arkasındaki arabalar gibi tek bir uzun süreci bekler.

ALGORİTMA 2

En Kısa İş Önce (SJF), Kesintisiz

İşlemci boşaldığında, bir sonraki CPU patlaması en kısa olan hazır süreci seç. Başlayan iş sonuna kadar çalışır.



Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	7	7	0	0
P2	2	4	12	10	6	6
P3	4	1	8	4	3	3
P4	5	4	16	11	7	7
Ortalama				**8**	**4**	**4**

Karar nasıl verildi?

- t = 0: yalnızca P1 hazır → P1 7'ye kadar çalışır.
- t = 7: P2 (4), P3 (1), P4 (4) bekliyor → en kısası P3.
- t = 8: P2 ile P4 4'te eşit → P2 önce gelmişti.
- Ortalama bekleme 4,75 (FCFS) → 4 ms.

ALGORİTMA 2B

Kesintili SJF: En Kısa Kalan Süre Önce (SRTF)

Yeni bir süreç geldiğinde, onun patlamasını çalışan sürecin kalan süresiyle karşılaştır. Kısa olan işlemciyi alır.



Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	16	16	9	0
P2	2	4	7	5	1	0
P3	4	1	5	1	0	0
P4	5	4	11	6	2	2
Ortalama				**7**	**3**	**0,5**

Her varışta verilen karar

- t = 2: P2'ye 4 gerekir < P1'in kalan 5'i → P1 kesilir.
- t = 4: P3'e 1 gerekir < P2'nin kalan 2'si → P2 kesilir.
- t = 5: P2 (2) < P4 (4) < P1 (5) → önce P2, sonra P4, sonra P1.
- Ortalama bekleme: 3 ms — günün en iyisi.

En Kısa İş Önce, Ortalama Beklemeyi Neden En Aza İndirir?

Bir yer değiştirme argümanı: planda art arda gelen iki işe bakın; uzun bir L, kısa bir S'den önce olsun.



Argüman

- Çiftin yerini değiştirmek, öncesindeki ve sonrasındaki işler için hiçbir şeyi değiştirmez.
- Çiftin içinde toplam bekleme L'den S'ye düşer.
- Uzun işin kısadan önce geldiği her plan iyileştirilebilir → patlama uzunluğuna göre sıralamak en iyisidir.

İki kesin ifade

- **SJF,** tüm işler aynı anda hazırsa en iyisidir (optimal).
- **SRTF,** işler farklı zamanlarda geliyorsa en iyisidir (bağlam değişimi maliyeti yok sayılırsa).

Püf noktası: zamanlayıcının bir sonraki CPU patlamasının uzunluğunu, süreç çalışmadan önce bilmesi gerekir.

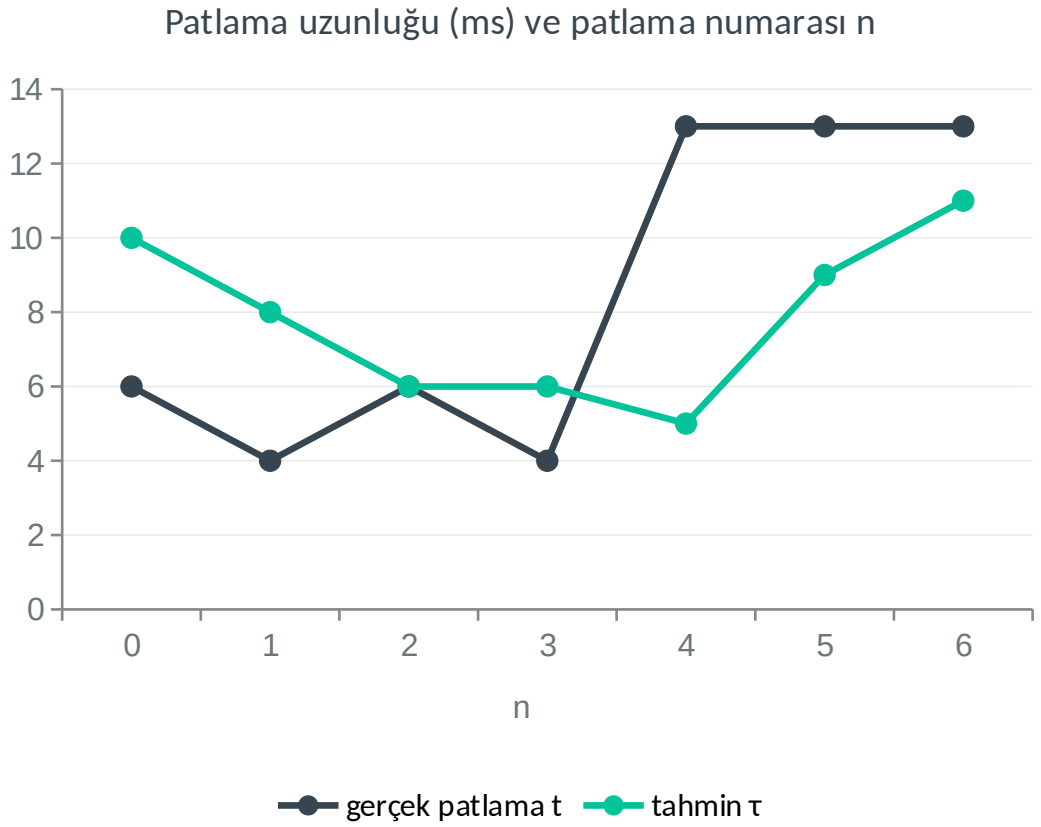
Bir Sonraki CPU Patlamasını Tahmin Etmek

İşletim sistemi geleceği bilemez, ama geçmişten tahmin yürütebilir: son patlamalar bir sonrakinin iyi göstergeleridir.

$$\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n$$

- t_n = n'inci patlamanın gerçek uzunluğu; τ_n = onun için yapılmış tahmin.
- **α** ($0 \le \alpha \le 1$) yakın geçmişin ağırlığı. Yaygın seçim: $\alpha = 1/2$.

```
Bir adım; α = 1/2, τ0 = 10 ms, t0 = 6 ms:
  τ1 = 0.5 · 6 + 0.5 · 10 = 8 ms
  τ2 = 0.5 · 4 + 0.5 ·  8 = 6 ms     (t1 = 4)
```



ALGORİTMA 3

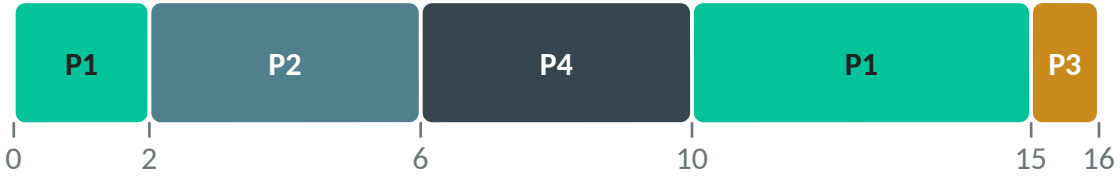
Öncelikli Zamanlama (Priority Scheduling)

Her sürecin bir öncelik numarası vardır; işlemci en yüksek öncelikli sürece (burada: en küçük numara) gider. SJF, öncelik = patlama uzunluğu olan özel durumdur.

Kesintisiz



Kesintili



Süreç	Öncelik	Bekleme (kesintisiz)	Bekleme (kesintili)
P1	3	0	8
P2	1	5	0
P3	4	11	11
P4	2	6	1
Ortalama		**5,5**	**5**

Dikkat edilecekler

- Kesintili: P2 (öncelik 1), t = 2'de gelir gelmez işlemciyi P1'den alır.
- P3'ün önceliği en düşüktür (4): iki sürümde de 11 ms bekler ve en son çalışır.
- Öncelikler iç (bellek ihtiyacı, patlama uzunluğu) ya da dış (önem, kimin ödediği) kaynaklı olabilir.

Açlık ve Çaresi: Yaşlandırma

Yoğun bir sistemde düşük öncelikli bir süreç süresiz bekleyebilir; aç kalır (starvation).

Sorun: açlık

- Daha yüksek öncelikli süreçlerin kesintisiz akışı işlemciyi meşgul tutar.
- Düşük öncelikli süreç hep hazırdır, ama hiç çalışmaz.
- Anlatılan bir hikâye: MIT 1973'te IBM 7094'ünü kapattığında, 1967'de gönderilmiş ve hiç çalışmamış düşük öncelikli bir iş bulmuş.

Çare: yaşlandırma (aging)

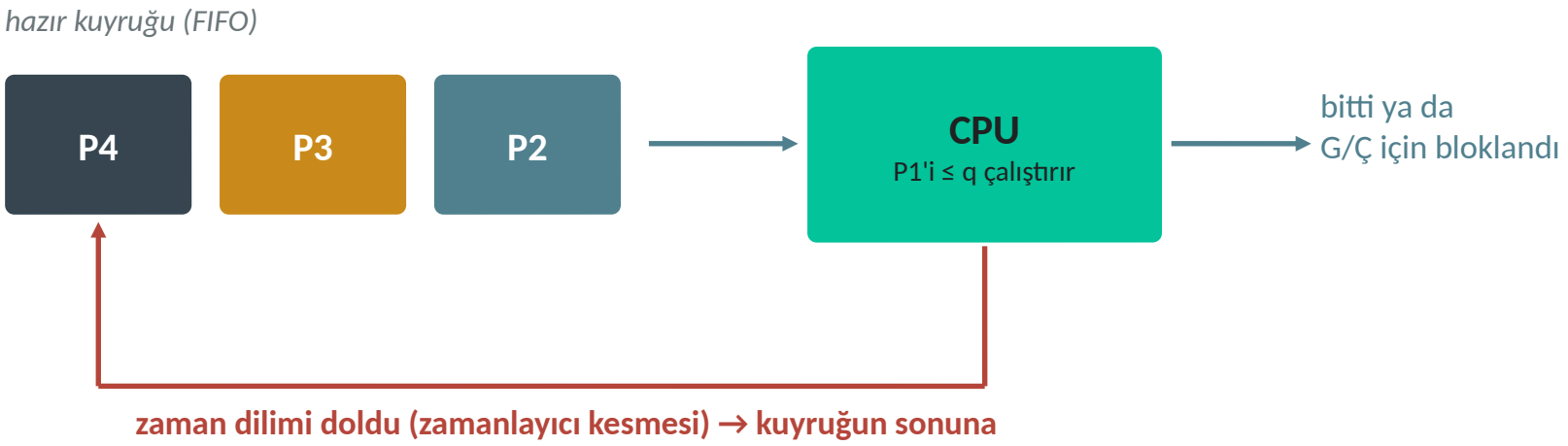
- Bir süreç ne kadar uzun beklerse önceliğini o kadar yükselt.
- Örnek: öncelikler 127 (düşük) … 0 (yüksek); her saniyelik bekleme önceliği 1 artırır.
- En düşük öncelikli süreç bile en geç 127 s = 2 dk 7 s sonra 0 önceliğine ulaşır.

Yaşlandırma 'belki hiçbir zaman'ı 'sınırlı bir süre içinde'ye çevirir. Aynı fikir 2. bölümde sınırlı bekleme olarak yeniden karşımıza çıkacak.

ALGORİTMA 4

Döngüsel Zamanlama (Round Robin, RR): Sırayla

FCFS artı zamanlayıcıyla kesme. Her süreç en çok bir zaman dilimi (quantum) q kadar çalışır, sonra kuyruğun sonuna gider.



- Kuyrukta n süreç varsa hiçbir süreç bir sonraki sırasından önce (n − 1) · q'dan fazla beklemez.
- Tipik zaman dilimleri: 10–100 ms; bir bağlam değişimi 10 µs'nin çok altında sürer.

Bizim kuralımız

- Bir zaman dilimi tam bittiği anda gelen süreç, kesilen süreçten önce kuyruğa girer.
- Patlama q'dan kısaysa süreç erken ayrılır ve sıradaki hemen başlar.

Çözümlü Örnek: q = 4 ms ile Döngüsel Zamanlama

Aynı dört süreç. Her zaman diliminden sonra hazır kuyruğunu izleyin (baş solda).



Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	12	12	5	0
P2	2	4	8	6	2	2
P3	4	1	9	5	4	4
P4	5	4	16	11	7	7
Ortalama				**8,5**	**4,5**	**3,25**

Hazır kuyruğu

- t = 4: kuyruk P2, P3, P1 (P2 ve P3 geldi, P1 sona döndü).
- t = 8: P2 bitti; kuyruk P3, P1, P4.
- t = 9: P3 1 ms sonra bitti; P1'e yalnızca 3 ms daha gerek.
- t = 12: P1 bitti; P4 16'ya kadar tek başına çalışır.

Aynı İşler, Daha Küçük Zaman Dilimi: q = 2 ms

Daha kısa sıralar: herkes işlemciye daha erken kavuşur, ama işlemci çok daha sık geçiş yapar.



Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	16	16	9	0
P2	2	4	9	7	3	0
P3	4	1	7	3	2	2
P4	5	4	15	10	6	4
Ortalama				**9**	**5**	**1,5**

q = 4 ile karşılaştırma

- Yanıt süresi düşer: ortalama 3,25 → 1,5 ms.
- Bekleme süresi artar: ortalama 4,5 → 5 ms.
- Bağlam değişimleri: 4 → 8.
- P1 dört parçaya bölünür ve t = 16'da en son biter.

Zaman Dilimini Seçmek

Zaman dilimi q	Ort. bekleme	Ort. yanıt	Geçiş
1 ms	5,5	0,75	14
2 ms	5	1,5	8
4 ms	4,5	3,25	4
8 ms	4,75	4,75	3

Bizim iş kümemiz, sched_check.py'den. q = 8 her patlamadan büyük ya da eşit olduğundan, RR tam olarak FCFS planını verir.

```
Geçişe giden zaman = s / (q + s)
geçiş maliyeti s = 0.1 ms ile:
  q =   1 ms  ->  9.1 %
  q =   4 ms  ->  2.4 %
  q =  10 ms  ->  1.0 %
  q = 100 ms  ->  0.1 %
```

q çok büyükse

RR, FCFS'ye dönüşür: etkileşimli kullanıcılar uzun işlerin arkasında bekler.

q çok küçükse

İşlemci zamanının büyük bir kısmı iş yapmaya değil, geçiş yapmaya gider.

Pratik kural: q'yu, CPU patlamalarının yaklaşık %80'i tek bir zaman dilimi içinde bitecek biçimde seçin; q, geçiş süresinden çok daha büyük olmalıdır.

Çok Düzeyli Kuyruklar ve Çok Düzeyli Geri Beslemeli Kuyruklar

Gerçek sistemler algoritmaları birleştirir: her birinin kendi politikası olan birkaç hazır kuyruğu.



Yeni bir süreç Kuyruk 0'a girer. Kısa, etkileşimli işler orada biter; uzun, CPU ağırlıklı işler dibe çöker.

İki türü

- **Çok düzeyli kuyruk:** süreç kalıcı olarak tek bir kuyruğa atanır (ör. ön plan = RR, arka plan = FCFS).
- **Çok düzeyli geri beslemeli kuyruk (MLFQ):** süreçler davranışlarına göre kuyruklar arasında yer değiştirir; çok uzun bekleyen yukarı çıkar (yaşlandırma).
- En genel çözüm, ama ayarlaması en zor olanı.

Linux Gerçekte Ne Yapıyor?

Linux sabit bir zaman dilimi kullanmaz. Çalışmaya hazır her göreve işlemciden adil bir pay vermeye çalışır.

Adil zamanlama

- **CFS** (Completely Fair Scheduler, 2007–2023): şimdiye kadar en az ağırlıklı işlemci zamanı almış görevi çalıştır.
- **EEVDF** (çekirdek 6.6'dan beri, 2023): aynı adalet hedefi, artı gecikmeye duyarlı görevler daha erken çalışsın diye sanal son tarihler.
- **nice değerleri** −20 (en çok işlemci) … +19 (en az). Her adım ağırlığı yaklaşık 1,25 kat değiştirir.
- **Gerçek zamanlı sınıflar** SCHED_FIFO ve SCHED_RR her zaman normal görevlerden önce çalışır.

```
$ taskset -c 1 ./cpu_hog &
$ taskset -c 1 nice -n 10 ./cpu_hog &
$ sleep 10
$ ps -o pid,ni,cls,pcpu,comm -C cpu_hog
  PID  NI CLS %CPU COMMAND
 7150   0  TS 88.0 cpu_hog
 7151  10  TS  9.4 cpu_hog
```

TEK işlemcide iki sonsuz döngü. Ağırlıklar 1024 (nice 0) ve 110 (nice 10): beklenen %90,3 : %9,7. 6.18 çekirdeğimizde ölçülen: %88,0 : %9,4.

1. BÖLÜMÜN ÖZETİ

Aynı İşler, Yedi Plan

Algoritma	Kesintili mi?	Ort. bekleme (ms)	Ort. dönüş (ms)	Ort. yanıt (ms)	Geçiş
FCFS	hayır	4,75	8,75	4,75	3
SJF	hayır	4	8	4	3
SRTF	evet	3	7	0,5	5
Öncelik (KS)	hayır	5,5	9,5	5,5	3
Öncelik (K)	evet	5	9	3	4
RR, q = 4	evet	4,5	8,5	3,25	4
RR, q = 2	evet	5	9	1,5	8

SRTF burada her sütunu kazanır — ama geleceği bilmesi gerekir. RR hiçbir bilgi istemez ve her beklemeyi (n − 1) · q ile sınırlar. FCFS en basitidir. Öncelik ortalamalara değil, öneme hizmet eder. (KS = kesintisiz, K = kesintili)

ARA

10 Dakika Ara

Sırada: iki iş parçacığı tek bir değişkeni paylaştığında neyin ters gittiği.

BÖLÜM 2

Süreç Eşzamanlaması

Zamanlayıcı iş parçacıklarını herhangi bir komutta değiştirebilir. Ya veri paylaşıyorlarsa?

SORUN

counter++ Tek Bir Adım Değildir

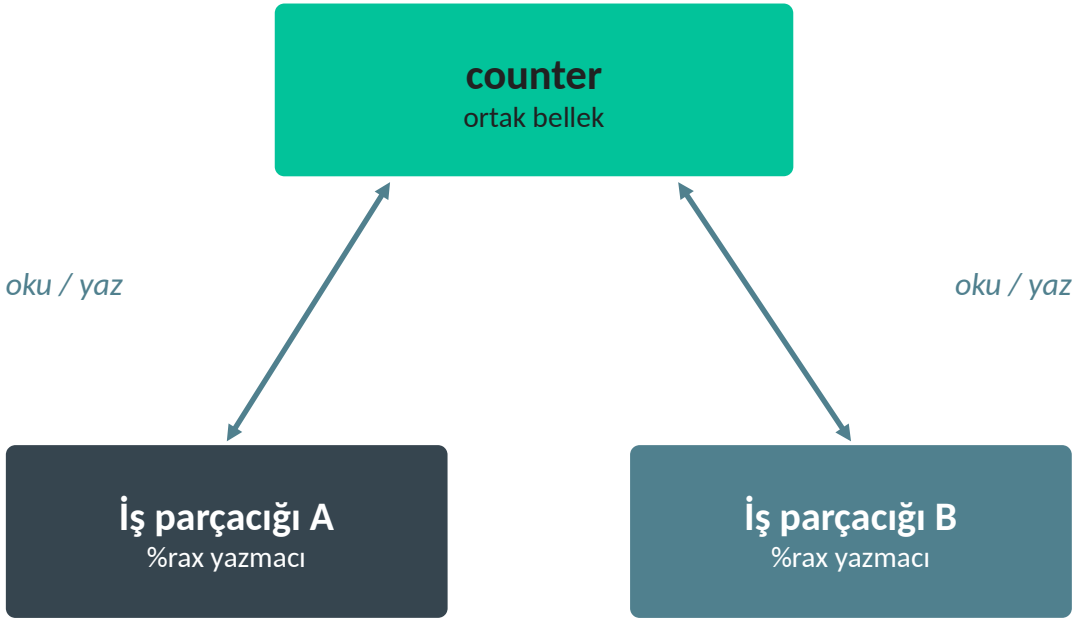
Tek satır C kodu üç makine komutuna dönüşür. Bu, race.c için gcc -O2 -S komutunun gerçek çıktısıdır (x86-64).

```
counter++;          // C kaynak kodu

movq  counter(%rip), %rax   # 1. OKU (LOAD)
addq  $1, %rax              # 2. 1 EKLE (ADD)
movq  %rax, counter(%rip)   # 3. YAZ (STORE)
```

Her iş parçacığının kendi yazmaçları vardır

- counter ortak bellekte durur; %rax her iş parçacığına özeldir (geçişte iş parçacığı denetim bloğuna kaydedilir).
- Bir geçiş — ya da başka bir işlemci — üç adımın herhangi ikisi arasına girebilir.



İki iş parçacığı, tek değişken, pek çok farklı sırayla iç içe geçebilen altı komut.

KAYIP BİR GÜNCELLEME

Şanssız Bir İç İçe Geçme

counter 5 ile başlıyor. A ve B iş parçacıkları counter++ işlemini birer kez yürütüyor. Doğru sonuç: 7.

Adım	İş parçacığı A	İş parçacığı B	A'nın %rax'ı	B'nin %rax'ı	counter (bellek)
1	counter'ı oku		5	–	5
2		counter'ı oku	5	5	5
3	1 ekle		6	5	5
4		1 ekle	6	6	5
5	yaz		6	6	6
6		yaz	6	6	6 × (7 olmalıydı)

B'nin yazması A'nınkinin üstüne yazar: bir artırma kaybolur. İki işlemcide 10 milyon kez çalıştırın, milyonlarcası kaybolur — birazdan canlı göreceğiz.

TANIM

Yarış Durumu (Race Condition)

Birden çok iş parçacığı aynı veriye eşzamanlı erişir, en az biri yazar ve sonuç yürütmenin tam sırasına bağlıdır.

Banka hesabı

İki ATM aynı anda aynı hesaptan para çeker; ikisi de eski bakiyeyi okur ve bir çekim kaybolur.

Uçaktaki son koltuk

İki rezervasyon sunucusu da '1 koltuk kaldı' görür ve ikisi de onu satar.

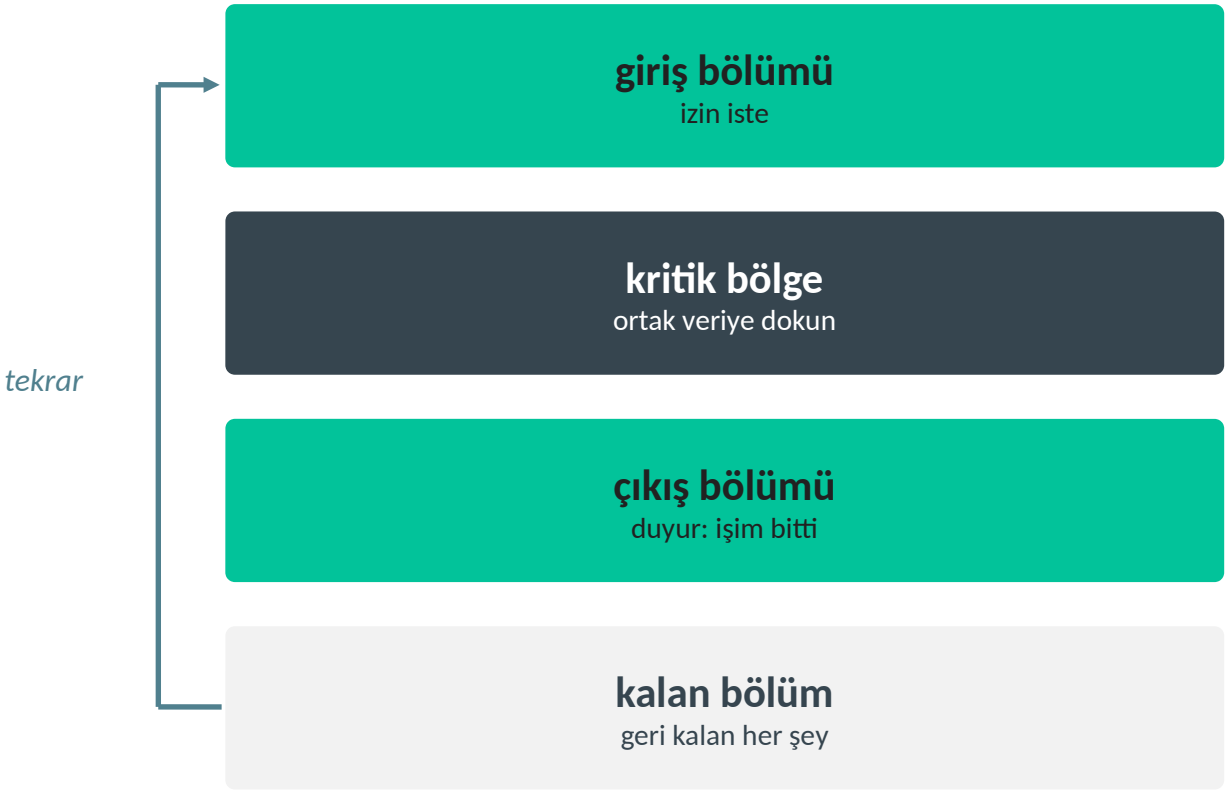
Çekirdeğin içinde

İki süreç aynı anda fork() çağırır; önlem alınmazsa ikisi de aynı yeni süreç kimliğini alabilir.

Yarış durumları zamanlama hatalarıdır: program bin testi geçip bir sonraki çalıştırmada çökebilir. Test ederek değil, tasarımla önleyin.

Kritik Bölgeler

Kritik bölge (critical section), bir iş parçacığının ortak veriye eriştiği koddur. İş parçacıkları onu asla aynı anda yürütmesin diye bir protokol tasarlayın.




```
while (true) {
    /* giriş bölümü     */
        counter++;   // kritik bölge
    /* çıkış bölümü     */
    /* kalan bölüm      */
}
```

Tasarlamamız gereken, giriş ve çıkış bölümleridir. Bugün bundan sonraki her şey — Peterson, atomik komutlar, muteksler, semaforlar — onları kurmanın bir yoludur.

Doğru Bir Çözüm için Üç Gereksinim

1. Karşılıklı dışlama

- Herhangi bir anda kritik bölgesinde en çok bir iş parçacığı bulunur.
- Bu, güvenlik özelliğidir: kayıp güncelleme imkânsız hâle gelir.

2. İlerleme

- Kritik bölge boşsa ve girmek isteyen iş parçacıkları varsa, biri kısa sürede içeri alınmalıdır.
- Kalan bölümündeki iş parçacıklarının veto hakkı yoktur.

3. Sınırlı bekleme

- Bir iş parçacığı girmek istedikten sonra, diğerleri ondan önce ancak sınırlı sayıda girebilir.
- Açlık yok — 1. bölümdeki yaşlandırmayla karşılaştırın.

Her iş parçacığının sıfırdan büyük bir hızla çalıştığını varsayarız; ama iş parçacıklarının göreli hızları ya da işlemci sayısı hakkında hiçbir varsayım yapmayız.

İLK FİKİR

Neden Kesmeleri Kapatmıyoruz?

Tek işlemcide bir iş parçacığı yalnızca bir kesmeyle işlemciden alınır. Kritik bölge boyunca kesme yok → geçiş yok → yarış yok.

İşe yaradığı yerler

- Tek işlemcili bir sistemde, çekirdeğin içinde.
- Çok kısa kritik bölgeler (birkaç komut).
- Erken UNIX çekirdekleri ve küçük gömülü sistemler bunu kullandı.

Neden ölçeklenmez?

- Çok işlemcili sistemler: diğer işlemciler çalışmayı sürdürür ve veriye yine dokunabilir.
- Kesmeleri tüm işlemcilerde kapatmak zaman alır ve bütün makineyi durdurur.
- Ayrıcalıklı komut: kullanıcı programları bunu yapamaz (yapmamalıdır da!).
- Uzun kritik bölgeler saati ve G/Ç aygıtlarını geciktirir.

Modern sistemler bunun yerine atomik donanım komutlarını kullanır — işlemciler arasında çalışırlar ve özel ayrıcalık gerektirmezler.

Peterson Çözümü (İki İş Parçacığı)

```
int  turn;          // bekleme sırası kimde?
bool flag[2];       // flag[i]: i girmek istiyor

/* i iş parçacığının kodu; j = 1 - i diğeri */
while (true) {
    flag[i] = true;          // girmek istiyorum ...
    turn = j;                // ... ama önce sen geç
    while (flag[j] && turn == j)
        ;                    // meşgul bekleme
    /* kritik bölge */
    flag[i] = false;         // işim bitti
    /* kalan bölüm */
}
```

Neden çalışır?

- Her iş parçacığı nazikçe sırayı diğerine verir.
- İkisi de girmek isterse turn yalnızca tek bir değer tutar — onu en son yazan bekler.
- Üç gereksinimi de sağlar (ispatı notlarda).
- **Ama:** modern işlemciler ve derleyiciler bellek erişimlerinin sırasını değiştirir; bu yüzden bellek bariyerleri olmadan gerçek donanımda başarısız olabilir. Bir fikir olarak öğrenin.

Atomik Komutlar: test_and_set

```
/* donanımın TEK ve bölünmez adımda yaptığı */
bool test_and_set(bool *target) {
    bool old = *target;
    *target = true;
    return old;
}

bool lock = false;            // ortak, false = boş

while (true) {
    while (test_and_set(&lock))
        ;                     // boşalana kadar dön
    /* kritik bölge */
    lock = false;             // bırak
    /* kalan bölüm */
}
```

Fikir

- Eski değeri oku ve true yaz — tek bir atomik adımda: arada başka hiçbir işlemci araya giremez.
- false döndü → kilit boştu ve artık benim.
- true döndü → kilidi başkası tutuyor; yeniden dene.
- Gerçek komutlar: x86'da xchg ve lock cmpxchg (compare-and-swap); ARM'da ldxr/stxr.

Muteks Kilitleri (Mutex Locks)

Programcılar test_and_set'i doğrudan kullanmaz. İşletim sistemi ve kütüphaneler onu iki işlemli bir kilide sarar.

```
acquire(&m);        // boşalana kadar bekle,
                    // sonra al
    /* kritik bölge */
release(&m);        // geri ver

/* POSIX threads (Linux, macOS) */
pthread_mutex_lock(&m);
    counter++;
pthread_mutex_unlock(&m);
```

Döner kilit (spinlock): meşgul bekleme

- Bekleyen iş parçacığı, kilit boşalana kadar işlemci yakarak döngüde döner.
- Çok işlemcide çok kısa kritik bölgeler için iyidir: bağlam değişimi gerekmez.

Uyuyan kilit

- Bekleyen iş parçacığı bloklanır, işlemci başkasını çalıştırır; release onu uyandırır.
- Uzun bölümler için daha iyidir. Linux pthread mutexleri futex kullanır: kilit boşken sistem çağrısı yok.

Semaforlar

Dijkstra (1965): yalnızca iki atomik işlemle değiştirilen bir S tamsayısı.

```
wait(S) {           // P() olarak da bilinir
    while (S <= 0)
        ;           // (ya da: bloklan)
    S--;
}
signal(S) {         // V() olarak da bilinir
    S++;            // (+ bekleyeni uyandır)
}
```

P ve V, Felemenkçe proberen (sınamak) ve verhogen (artırmak) sözcüklerinden gelir. POSIX'te: sem_wait() ve sem_post().

İkili semafor (0 ya da 1)

- 1 ile başlarsa: muteks kilidi gibi davranır.
- 0 ile başlarsa: sıra dayatır — B'deki S2, ancak A S1'in bittiğini bildirdikten sonra çalışır.

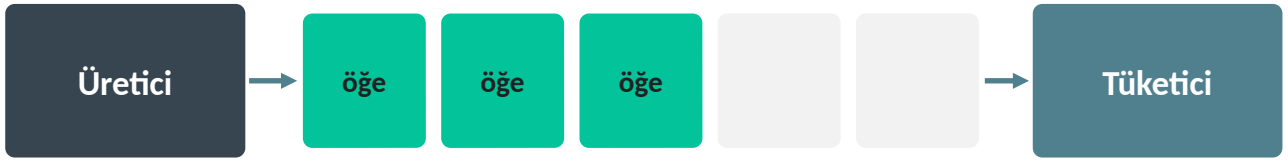
Sayan semafor (0 … N)

- N = özdeş kaynak sayısıyla başlar, ör. 3 yazıcı.
- Her kullanıcı önce bekler (S−−), sonra bildirir (S++); 4. kullanıcı bir yazıcı boşalana kadar bloklanır.

KLASİK PROBLEM

Sınırlı Tampon: Üretici ve Tüketici

Üretici n gözlü bir tampona öğe koyar; tüketici onları alır. Üç semafor ikisini eşgüdümler.



n = 5 göz: burada full = 3, empty = 2

Semafor	Başlangıç değeri	Saydığı
`empty`	n	boş gözler
`full`	0	dolu gözler
`mutex`	1	tampon erişim kilidi

Üretici empty = 0 olunca (tampon dolu), tüketici full = 0 olunca (tampon boş) bloklanır.

```
/* üretici */
wait(empty);
wait(mutex);
 öğe ekle
signal(mutex);
signal(full);
```

```
/* tüketici */
wait(full);
wait(mutex);
 öğe çıkar
signal(mutex);
signal(empty);
```

Sıra önemlidir: önce sayan semaforda, sonra mutex'te bekleyin. İkisinin yerini değiştirmek kilitlenmeye yol açabilir (sonraki slayt).

BİR UYARI

Dikkatsiz Kilitleme Kilitlenmeye Yol Açabilir

S ve Q adlı iki semafor, ikisi de 1 ile başlıyor. Her iş parçacığına ikisi de gerekiyor — ama onları farklı sırayla alıyorlar.

```
/* P0 iş parç. */
wait(S);
wait(Q);
  ...
signal(S);
signal(Q);
```

```
/* P1 iş parç. */
wait(Q);
wait(S);
  ...
signal(Q);
signal(S);
```

Zaman	P0	P1	S	Q
1	wait(S) ✓		0	1
2		wait(Q) ✓	0	0
3	**wait(Q) bloklanır**		0	0
4		**wait(S) bloklanır**	0	0

Kilitlenme (deadlock): her iş parçacığı yalnızca diğerinin tetikleyebileceği bir olayı bekler — ikisi de sonsuza dek bekler. En basit çare: tüm iş parçacıkları kilitleri aynı genel sırayla alsın. Aynı tuzak: tampon doluyken empty'den önce mutex'i alan bir üretici. (Kilitlenmenin ayrıntıları bu dersin kapsamı dışındadır.)

UYGULAMA

Örnek 1: C'de Gerçek Bir Yarış (race.c)

```c
#define N 10000000L        /* her biri için */
volatile long counter = 0;     /* ORTAK      */

void *worker(void *arg)
{
    for (long i = 0; i < N; i++)
        counter++;             /* atomik DEĞİL */
    return NULL;
}

int main(void)
{
    pthread_t a, b;
    pthread_create(&a, NULL, worker, NULL);
    pthread_create(&b, NULL, worker, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %8ld of %ld ...\n",
           counter, 2 * N);
}
/* programın tamamı: code/race.c */
```

Deney

- İki iş parçacığı, her biri on milyon kez 1 ekliyor → 20.000.000 vermeli.
- Güncellemelerin %28–40'ı kayboluyor — iki iş parçacığı TEK bir işlemciyi paylaşsa bile.
- Aynı program, her çalıştırmada farklı yanıt:

```
$ ./race          # run 4 times
counter = 12250428 of 20000000  (lost 7749572)  0.052 s
counter = 12005053 of 20000000  (lost 7994947)  0.100 s
counter = 13797565 of 20000000  (lost 6202435)  0.110 s
counter = 12046532 of 20000000  (lost 7953468)  0.080 s
$ taskset -c 0 ./race   # pinned to ONE CPU
counter = 14324099 of 20000000  (lost 5675901)  0.034 s
counter = 12867601 of 20000000  (lost 7132399)  0.034 s
```

Örnek 2: Çözüm ve Bedeli (race_fixed.c)

```
long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *arg)
{
    for (long i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);    /* giriş */
        counter++;                    /* kritik
                                         bölge */
        pthread_mutex_unlock(&lock);  /* çıkış */
    }
    return NULL;
}
/* main() aynı; programın tamamı:
   code/race_fixed.c */
```

Doğru, ama daha yavaş

- Her zaman tam olarak 20.000.000.
- Kilitsiz (yanlış): 0,05–0,11 s. Kilitli (doğru): 0,75–1,12 s — yaklaşık 10–20 kat yavaş.
- Neden? 20 milyon kilitle/aç çifti ve iki işlemci kilit için çekişiyor.

```
$ ./race_fixed    # run 3 times
counter = 20000000 of 20000000  (lost      0)  1.122 s
counter = 20000000 of 20000000  (lost      0)  1.115 s
counter = 20000000 of 20000000  (lost      0)  0.749 s
```

Örnek 3: POSIX Semaforlarıyla Üretici–Tüketici

```
sem_t empty, full, mutex;   /* SIZE, 0, 1 */

void *producer(void *arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);   /* boş göz?    */
        sem_wait(&mutex);
        buffer[in] = item;  in = (in + 1) % SIZE;
        sem_post(&mutex);
        sem_post(&full);    /* bir öğe daha */
    }
    return NULL;
}
void *consumer(void *arg) {
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);    /* öğe var mı? */
        sem_wait(&mutex);
        item = buffer[out]; out = (out + 1) % SIZE;
        sem_post(&mutex);
        sem_post(&empty);   /* bir göz daha  */
        sum += item;   /* ... her 5.'yi yazdır */
    }
}
```

Sözde koddan C'ye

- wait → sem_wait, signal → sem_post.
- sem_init(&empty, 0, SIZE): 0, 'tek bir sürecin iş parçacıkları arasında ortak' demektir.
- 5 gözlü tampon, 20 öğe: üretici 5 öğe öne geçtiğinde bloklanır.

```
$ ./prodcons
consumed  5   running sum  15
consumed 10   running sum  55
consumed 15   running sum 120
consumed 20   running sum 210
sum of 1..20 = 210 (expected 210)
```

KISA SINAV

Kendinizi Sınayın

1. Patlamaları 5, 2, 8 ms olan işlerin hepsi t = 0'da geliyor. SJF sırasını ve ortalama bekleme süresini verin.

2. Üstel ortalamada α = 0,5, τ = 8 ms ve son patlama t = 12 ms. Bir sonraki tahmin nedir?

3. Hepsi 50 ms'den az süre isteyen işler üzerinde q = 100 ms ile döngüsel zamanlama hangi algoritma gibi davranır? Neden?

4. Bir kritik bölge çözümünün üç gereksinimini sayın. 'Yalnızca A iş parçacığı girebilir' kuralı hangisini ihlal eder?

KAPANIŞ

5. Hafta: Temel Çıkarımlar

- **FCFS** basittir ama konvoy etkisinden muzdariptir (aynı işler için ortalama bekleme 17 ya da 3 ms).
- **SJF / SRTF** ortalama bekleme süresini en aza indirir, ama bir sonraki patlama uzunluğunu bilmeleri gerekir — üstel ortalamayla tahmin edilir.
- **Öncelikli** zamanlama düşük öncelikli işleri aç bırakabilir; yaşlandırma bunu düzeltir.
- **Döngüsel zamanlama** her beklemeyi $(n - 1) \cdot q$ ile sınırlar; q çok büyük → FCFS, q çok küçük → geçiş yükü. Linux adil pay hedefler (CFS, 6.6'dan beri EEVDF).
- **Yarış durumu:** counter++ oku–ekle–yaz demektir; iç içe geçmeler güncelleme kaybettirir (testimizde milyonlarca).
- **Kritik bölgeler** karşılıklı dışlama, ilerleme ve sınırlı bekleme ister; atomik komutlardan muteks kilitleri ve semaforlar kurulur.

Ara sınav (8. hafta) için: yukarıdaki her algoritmada Gantt şeması çizip bekleme, dönüş ve yanıt sürelerini hesaplayabilmelisiniz.

GELECEK HAFTA

Bellek Yönetimi, Sayfalama ve Bölütleme

6. haftanın konuları:

- Adres bağlama; mantıksal ve fiziksel adresler; MMU
- Takas (swapping) ve bitişik bellek tahsisi
- Dış ve iç parçalanma (fragmentation)
- Sayfalama, sayfa tabloları, TLB ve etkin erişim süresi; bölütleme

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz